MedTech
Mediterranean
Institute of Technology

CS202 – ADVANCED PROGRAMMING

JAVA DATABASE CONNECTIVITY

SOPHOMORE SE

SPRING 2026

Agenda

1. Relational databases – brief review
2. JDBC: definition, architecture, classes & interfaces
3. Steps of DB connecting
4. Practical examples

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Relational Databases – brief review

Relational DB :

- Set of tables or relations. It is a type of DB that stores and provides access to data points that are related to one another.
- are based on the relational model, an intuitive, straightforward way of representing data in tables.
- each row in the table is a record with a unique ID called the key.
- columns of the table hold attributes of the data, and each record usually has a value for each attribute, making it easy to establish the relationships among data points.

DBMS : Software tool allowing to manage databases :

- Create and modify tables
- Request databases
- Ensuring data securtiy & integrity
- Manage transactions and concurrent access

Transactions : A logical processing unit that, when applied to a consistent state of the database, renders a new, consistent but modified state of the database.

SQL (Structured Query Language) : Language for handling data

ODBC : Open DataBase Connectivity: A Microsoft interface to access to SQL DBs.

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

Relational Database

StudentCourses

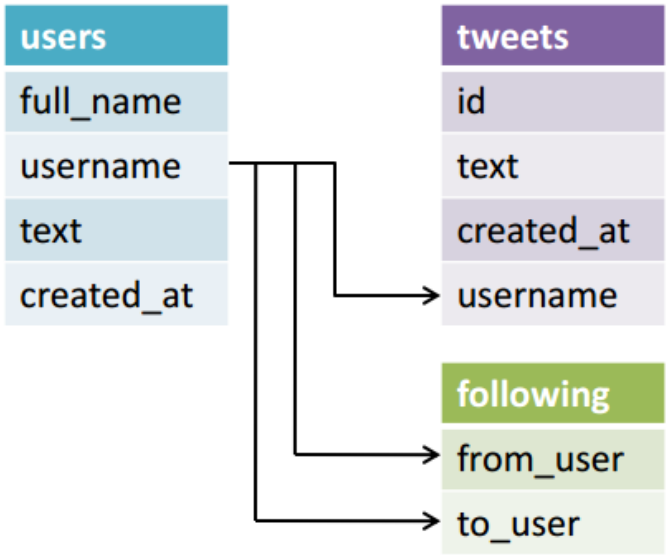
StudentId	CourseId
1	1
1	2
1	3
2	3
2	4
3	3
3	5

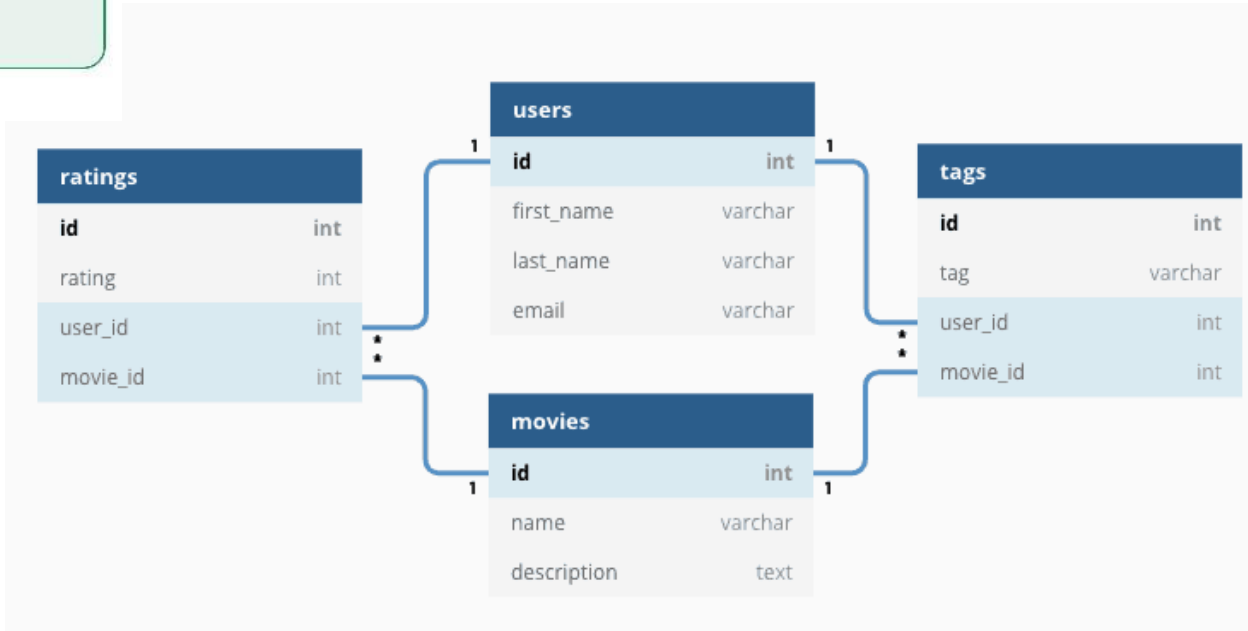
Students

Id	Name
1	Sam
2	Mary
3	Tine

Courses

Id	Name
1	SQL Server
2	ASP.NET MVC
3	MongoDB
4	Java
5	PHP





What is JDBC?

JDBC (Java Database Connectivity) is the **Java API** (Application programming Interface) that manages connecting to a database, issuing queries and commands, and handling result sets obtained from the database.

It is an API allowing uniform access to relational databases:

- **portable on most OS**
- **independent of DBMS** (only the connection phase is specific – appropriate driver)
- **compatible with most DBMSs:** Oracle, Postgresql, MySQL, Informix, Sybase, MS SQL Server, …

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

JDBC API

- Developed as an alternative to the C-based ODBC (Open Database Connectivity) API, JDBC offers a programming-level interface that handles the mechanics of Java applications communicating with a relational database or RDBMS.

- Part of the JDK (Java Development Kit)

History

- Released as part of JDK 1.1 in 1997, JDBC was one of the first components developed for the Java persistence layer.
- Initially conceived as a client-side API, enabling a Java client to interact with a data source.
- Changed with JDCB 2.0, which included an optional package supporting server-side JDBC connections.
- Every new JDBC release since then has featured updates to both the client-side package (java.sql) and the server-side package (javax.sql).
- JDBC 4.3, the most current version as of this writing, was released as part of Java SE 9 in September 2017.

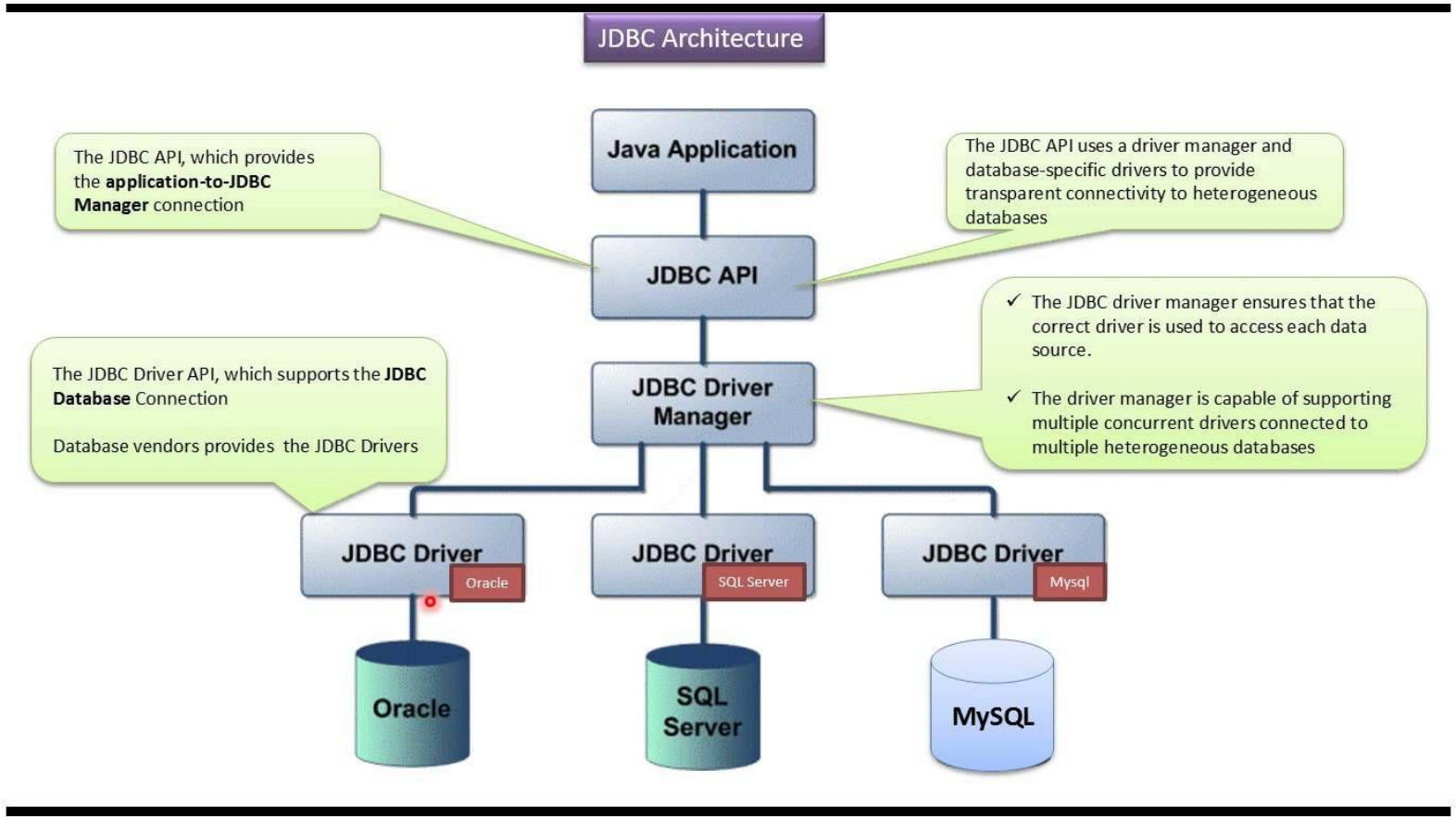
SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

JDBC API – architecture

2 layers:

1. **JDBC API** supports communication between Java application and JDBC manager.
   - visible layer, necessary to interface Java applications and DBMS
   - package java.sql

2. **JDBC driver** supports communication between JDBC manager and DB driver.
   - driver's implementation
   - driver layout interface between low-level access to the DBMS engine and application



Drivers

- each DBMS uses a specific driver
- allows us to translate JDBC requests into the DBMS language
- made up of classes implementing certain interfaces of java.sql
- more than 200 drivers are currently available

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

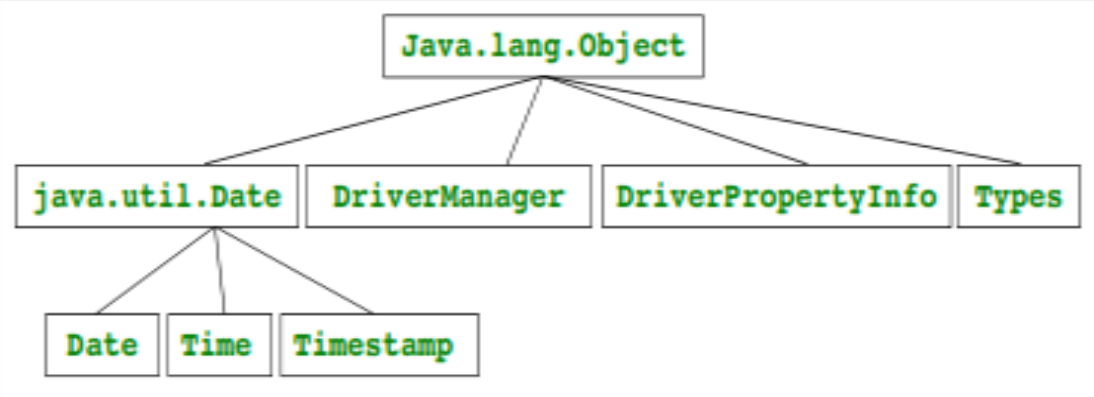
JDBC API – package, classes & interfaces

- Java API **classes** and **interfaces** are in the **java.sql package** (import java.sql.*) and this is completed by the **javax.sql package**

- **java.sql package** is based on **8 classes** and **8 interfaces** defining the necessary objects:
  - to connect to a remote DB
  - to create and execute of SQL queries
  - to retrieve and process results

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

Classes of java.sql package



DriverManager **:** manages drivers and launch connections to DB

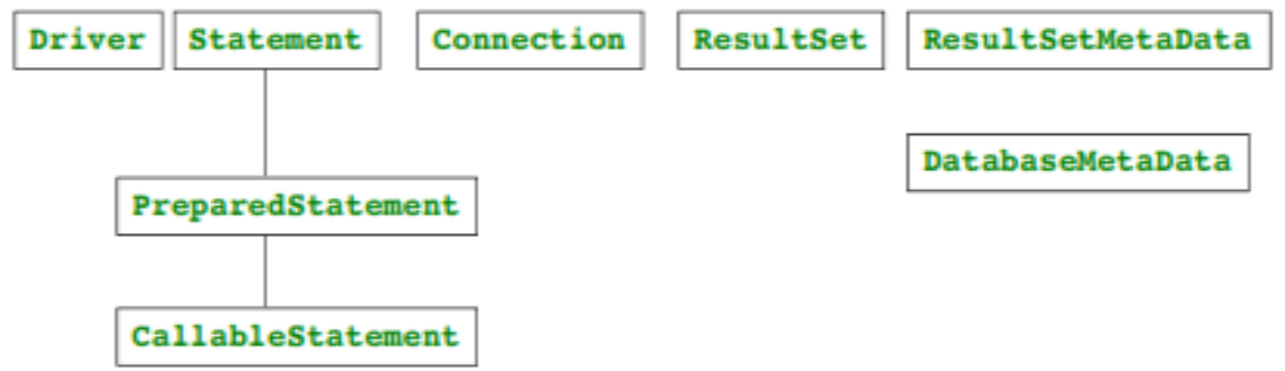
Date **:** SQL date

Time **:** hours, minutes, seconds in SQL

TimeStamp : like Time (micro-second precision)

Types : constants

Interfaces of java.sql package



Driver : retunrs an instance of connection ***Connection*** to a DB

Statement : SQL instruction

PreparedStatement: parameterized SQL instruction

CallableStatement : procedure stored in the DB

ResultSet : tuples retrieved by an SQL statement

ResultSetMetaData : description of the tuples retrieved

DatabaseMetaData : DB information

Steps for connecting to a DB with JDBC

Driver Registration

BD Connection

Request creation

Request execution

Request handling

Closing the connection

- ✓ Install or locate the database to be handled
- ✓ Include the JDBC library and make sure that the classpath variable related to the required JDBC driver is configured
- ✓ Close the connection when you're finished.

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

DB connecting – 1st step

▪ Import JDBC packages

Add **import** statements to the Java program to import required classes.

The **Import** statements tell the Java compiler where to find the classes referenced in the code and are placed at the very beginning of the source code.

To use the standard JDBC package, which allows us to select, insert, update, and delete data in SQL tables, add the following *imports* to the source code

```
import java.sql.* ; // for standard JDBC programs
import java.math.* ; // for BigDecimal and BigInteger support
```

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

DB connecting – 1st step

- Import JDBC packages
- **Specify the driver type to be used and register JDBC driver**

- This step causes the JVM to load the desired driver implementation into memory so it can fulfill the JDBC requests.
- It is required to register the driver in the program before use it (to be done only once).
- Registering the driver is the process by which the Oracle driver's class file is loaded into the memory, so it can be utilized as an implementation of the JDBC interfaces.

The most common approach to register a driver is to use Java's **Class.forName()** method, to dynamically load the driver's class file into memory, which automatically registers it.
This method is preferable because it makes the driver registration **configurable and portable**.

```
try {
        Class.forName("oracle.jdbc.driver.OracleDriver");
} catch(ClassNotFoundException ex)
        { System.out.println("Error: unable to load driver class!");
          System.exit(1); }
```

Other examples of drivers

Class.forName("oracle.jdbc.driver.OracleDriver");
Class.forName("com.mysql.jdbc.Driver");
Class.forName("org.postgresql.Driver");

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

use getInstance() method to work around noncompliant JVMs

```
try {
        Class.forName("oracle.jdbc.driver.OracleDriver").newInstance();
}
   catch(ClassNotFoundException ex)
        {       System.out.println("Error: unable to load driver class!");
                System.exit(1); }
   catch(IllegalAccessException ex)
        {       System.out.println("Error: access problem while loading!");
                System.exit(2); }
   catch(InstantiationException ex)
        {       System.out.println("Error: unable to instantiate driver!");
                System.exit(3); }
```

DB connecting – 1st step

- Import JDBC packages
- Specify the driver type to be used and register JDBC driver
- **Database URL Formulation**

- This is to create a properly formatted address that points to the database to which we will connect.
- After loading the driver, we can establish a connection using the **DriverManager.getConnection()** method.

```
  - getConnection(String url)
  - getConnection(String url, Properties prop)
→ getConnection(String url, String user, String password)
```

- A database URL is an address that points to the database to be handled.
- Formulating a database URL is where most of the problems associated with establishing a connection occurs.

RDBMS	JDBC driver name	URL format
MySQL	com.mysql.jdbc.Driver	**jdbc:mysql:**//hostname/ databaseName
ORACLE	oracle.jdbc.driver.OracleDriver	**jdbc:oracle:thin:@**hostname:port Number:databaseName
DB2	COM.ibm.db2.jdbc.net.DB2Driver	**jdbc:db2:**hostname:port Number/databaseName
Sybase	com.sybase.jdbc.SybDriver	**jdbc:sybase:Tds:**hostname: port Number/databaseName

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

DB connecting – 2nd step

- **Establish a connection to the specified DB**

Create Connection Object − code a call to the ***DriverManager object's*** getConnection( ) method to establish a database connection.

```
Connection connexion = DriverManager.getConnection(url, userName, password);
```

This method throws a ***SQLException***

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

DB connecting – 2nd step

- Establish a connection to the specified DB
- **Using a Database URL with a username and password**

Create Connection Object − code a call to the ***DriverManager object's*** getConnection( ) method to establish a database connection.

The most commonly used form of **getConnection()** requires you to pass a database **URL**, a **username**, and a **password**

Oracle case using Oracle's **thin** driver
host:port:databaseName value for the database portion of the **URL**.
host at TCP/IP address 192.0.0.1 with a host name of myownhost,
Oracle listener is configured to listen on port 1521,
DB name is EMP,

➔ complete database URL would be `jdbc:oracle:thin:@myownhost:1521:EMP`

```
String URL = "jdbc:oracle:thin:@myownhost:1521:EMP";
String USER = "username";
String PASS = "password";
Connection c = DriverManager.getConnection(URL, USER, PASS);
```

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

DB connecting – 3rd step

▪ Create a request

Based on the established connection, it is possible to create a request corresponding to SQL query or instruction.
This can be done by using ***Statement*** interface

There are three types of ***Statement*** :

- **Statement** : simple instruction (simple static queries)

```
Statement q1 =  c.createStatement(str);
```

- **PreparedStatement :** precompiled parameterizable (or dynamic) queries (queries with input / output parameters)

```
PreparedStatement q2 = c.prepareStatement(str);
```

- **CallableStatement** : stored procedures

```
CallableStatement q3 = c.prepareCall(str);
```

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

DB connecting – 4th step

- **Execute a request**

Based on the established connection, it is possible to create a request corresponding to SQL query or instruction.
This can be done by using ***Statement*** interface

Simple query : Statement

```
Statement st =  connexion.createStatement();
ResultSet executeQuery(String st)  //st is the SQL query to be executed
ResultSet executeUpdate(String st)
```

- Executes a ***SELECT*** query on the DB
- Returns an object of ***ResultSet*** type containing all tuples satisfying the specified conditions in the where clause.

```
int executeUpdate(String ordre)
```

- Executes an ***INSERT***, ***UPDATE***, ou ***DELETE*** query

```
void close ()
```

```
Statement stmt = connexion.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM Author WHERE aut_id > 60");
```

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

DB connecting – 5th step

- **Retrieve and handle results**

Based on the established connection, it is possible to create a request corresponding to SQL query or instruction.
This can be done by using ***Statement*** interface

In the returned results, columns are identified by their n° (starting from 1) or by their name.

Access to coulmns / data of a line returned as result can be done thanks to the method

```
[type] get[Type](int col)
```

- Returns the content the result of the corresponding column having as data type **[type]** like String, int, float, boolean …

```
String getString(int col)
```

```
int id = rs.getInt("auth_id");
String Fname = rs.getString("auth_FName");
String Lname = rs.getString(3);
```

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues

The **ResultSet** interface provides methods for retrieving and manipulating the results of executed queries, and **ResultSet** objects can have different functionalities and characteristics.
These characteristics are **type**, **concurrency**, and **cursor holdability**.

The sensitivity of a ResultSet object is determined by one of **three different ResultSet types**:

TYPE_FORWARD_ONLY: The result set cannot be scrolled; its cursor moves forward only, from before the first row to after the last row. The rows contained in the result set depend on how the underlying database generates the results. That is, it contains the rows that satisfy the query at either the time the query is executed or as the rows are retrieved.

TYPE_SCROLL_INSENSITIVE: The result can be scrolled; its cursor can move both forward and backward relative to the current position, and it can move to an absolute position. The result set is insensitive to changes made to the underlying data source while it is open. It contains the rows that satisfy the query at either the time the query is executed or as the rows are retrieved.

TYPE_SCROLL_SENSITIVE: The result can be scrolled; its cursor can move both forward and backward relative to the current position, and it can move to an absolute position. The result set reflects changes made to the underlying data source while the result set remains open.

The default ResultSet type is TYPE_FORWARD_ONLY.

Note: Not all databases and JDBC drivers support all ResultSet types.
The method **DatabaseMetaData.supportsResultSetType** returns true if the specified ResultSet type is supported and false otherwise.

The **ResultSet** interface provides methods for retrieving and manipulating the results of executed queries, and **ResultSet** objects can have different functionalities and characteristics.
These characteristics are **type**, **concurrency**, and **cursor holdability**.

The concurrency of a ResultSet object determines what level of update functionality is supported. There are **two concurrency levels**:

CONCUR_READ_ONLY: The ResultSet object cannot be updated using the ResultSet interface.

CONCUR_UPDATABLE: The ResultSet object can be updated using the ResultSet interface.

The default ResultSet concurrency is CONCUR_READ_ONLY.

Note: Not all JDBC drivers and databases support concurrency.
The method **DatabaseMetaData.supportsResultSetConcurrency** returns true if the specified concurrency level is supported by the driver and false otherwise.

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

The **ResultSet** interface provides methods for retrieving and manipulating the results of executed queries, and **ResultSet** objects can have different functionalities and characteristics.
These characteristics are **type**, **concurrency**, and **cursor holdability**.

Calling the method **Connection.commit** can close the ResultSet objects that have been created during the current transaction. In some cases, however, this may not be the desired behavior.

The ResultSet property holdability gives the application control over whether ResultSet objects (cursors) are closed when commit is called.

The following ResultSet constants may be supplied to the Connection methods **createStatement**, **prepareStatement**, and **prepareCall**:

HOLD_CURSORS_OVER_COMMIT: ResultSet cursors are not closed; they are holdable: they are held open when the method commit is called. Holdable cursors might be ideal if your application uses mostly read-only ResultSet objects.

CLOSE_CURSORS_AT_COMMIT: ResultSet objects (cursors) are closed when the commit method is called. Closing cursors when this method is called can result in better performance for some applications.

The default cursor holdability varies depending on your DBMS.

Note: Not all JDBC drivers and databases support holdable and non-holdable cursors.
The following method, **JDBCTutorialUtilities.cursorHoldabilitySupport**, outputs the default cursor holdability of ResultSet objects and whether **HOLD_CURSORS_OVER_COMMIT** and **CLOSE_CURSORS_AT_COMMIT** are supported.

Cursors

We can access the data in a **ResultSet** object through a **cursor**, which points to one row in the ResultSet object.

However, when a ResultSet object is first created, the cursor is positioned before the **first row**.

We can move the cursor by calling the **ResultSet.next** method.

There are other methods available to move the cursor:

- **next:** Moves the cursor forward one row. Returns true if the cursor is now positioned on a row and false if the cursor is positioned after the last row.
- **previous:** Moves the cursor backward one row. Returns true if the cursor is now positioned on a row and false if the cursor is positioned before the first row.
- **first:** Moves the cursor to the first row in the ResultSet object. Returns true if the cursor is now positioned on the first row and false if the ResultSet object does not contain any rows.
- **last:** Moves the cursor to the last row in the ResultSet object. Returns true if the cursor is now positioned on the last row and false if the ResultSet object does not contain any rows.
- **beforeFirst:** Positions the cursor at the start of the ResultSet object, before the first row. If the ResultSet object does not contain any rows, this method has no effect.
- **afterLast:** Positions the cursor at the end of the ResultSet object, after the last row. If the ResultSet object does not contain any rows, this method has no effect.
- **relative(int rows):** Moves the cursor relative to its current position.
- **absolute(int row):** Positions the cursor on the row specified by the parameter row.

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

DB connecting – 6th step

- **Close JDBC connection**

Create Connection Object − code a call to the ***DriverManager object's*** getConnection( ) method to establish a database connection.
At the end of the JDBC program, it is required explicitly to close all the connections to the database to end each database session with the **close()** method
However, if you forget, Java's garbage collector will close the connection when it cleans up stale objects (bad practice)
Closing open connections frees up mobilized resources.
It is possible to provide a **'finally'** block in the code . A finally block always executes, regardless of an exception occurs or not.

```
MyConnection.close();
MyResultSet.close();
MyStatement.close();
```

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB Business | MedTech Ingéniorat | LCI Langues